

Milestone 3: RISC-V Assembly Kalman Filter Implementation

Linear and Extended Kalman Filters in Scalar RISC-V Assembly

Fatima Faisal
Shehryar Tahir Awan
Raahim Imran
Muhammad Ibrahim

1 Introduction

Milestone 3 reimplements both the Linear Kalman Filter (LKF) and the Extended Kalman Filter (EKF) from Milestone 2 entirely in RISC-V scalar assembly (LP64D calling convention). The goal is to demonstrate that hand-written assembly can replicate, and in the EKF case improve upon, the numerical accuracy of the reference C++ implementation while exposing the low-level register management, stack discipline, and floating-point instruction selection that underpin performance-critical numerical code.

Both filters operate on a **single 276-dimensional state vector** $\mathbf{x} \in \mathbb{R}^{276}$ that concatenates the 12-dimensional kinematic state of all 23 joints simultaneously:

$$\mathbf{x} = [\mathbf{x}_{\text{pelvis}}^T \mid \mathbf{x}_{L5}^T \mid \cdots \mid \mathbf{x}_{\text{toeLeft}}^T]^T,$$
$$\mathbf{x}_j = [p_x, v_x, a_x, j_x, p_y, v_y, a_y, j_y, p_z, v_z, a_z, j_z]^T \in \mathbb{R}^{12}.$$

The system matrices $F \in \mathbb{R}^{276 \times 276}$, $Q \in \mathbb{R}^{276 \times 276}$, $H \in \mathbb{R}^{69 \times 276}$, and $R \in \mathbb{R}^{69 \times 69}$ are block-diagonal, with one 12×12 (or 3×12 , 3×3) block per joint. A *single* call to `lkf_predict` / `lkf_update` processes all 23 joints at once per timestep. The LKF uses a linear measurement matrix H ; the EKF linearises a nonlinear spherical measurement function $h(\mathbf{x})$ via an analytic 69×276 block-diagonal Jacobian J_k computed per timestep in assembly.

2 Architecture and Calling Convention

All functions target the RISC-V LP64D ABI. Key conventions used throughout `lkf_asm.s` and `ekf_asm.s`:

- Integer arguments arrive in `a0`–`a7`; arguments beyond eight are pushed on the stack by the caller.
- Floating-point arguments arrive in `fa0`–`fa7`.
- `s0`–`s11` and `fs0`–`fs11` are callee-saved; every non-leaf function saves and restores exactly the registers it uses.
- `t0`–`t6` and `ft0`–`ft11` are caller-saved temporaries, free to clobber across calls.
- The return address `ra` is saved to the stack before any call instruction.
- All matrices are stored row-major as 64-bit IEEE-754 doubles (8 bytes each). Element $[i][j]$ of an $r \times c$ matrix at base pointer B is at byte offset $(i \cdot c + j) \cdot 8$.

3 Matrix Primitive Functions

Nine matrix primitives are shared between the LKF and EKF assemblies. Each is a standalone callable function that obeys the ABI.

3.1 `mat_zero` and `mat_eye`

`mat_zero(A, r, c)` zeroes all $r \cdot c$ elements with a single `fmv.d.x ft0, zero / fsd loop`. Being a pure leaf function, it requires no stack frame.

`mat_eye(A, n)` calls `mat_zero` first, then writes 1.0 to diagonal elements by advancing the pointer by $(n + 1) \cdot 8$ bytes per iteration. The value 1.0 is materialised via `li t0, 1` followed by `fcvt.d.w ft1, t0`, since RISC-V has no immediate-to-FP-register instruction.

Table 1: Register allocation for `mat_zero` and `mat_eye`

Register	<code>mat_zero</code>	<code>mat_eye</code>
<code>a0</code>	base ptr (advances)	base ptr / arg to <code>mat_zero</code>
<code>a1</code>	rows r	dimension n
<code>a2</code>	cols c	$c = n$ (for <code>mat_zero</code> call)
<code>a3</code>	element counter $r \cdot c$	–
<code>a4</code>	–	diagonal loop counter
<code>s0</code>	–	saved base pointer
<code>s1</code>	–	saved n
<code>t0</code>	–	integer 1 (for <code>fcvt</code>)
<code>t1</code>	–	diagonal stride $(n + 1) \times 8$
<code>ft0</code>	constant 0.0	constant 0.0
<code>ft1</code>	–	constant 1.0

3.2 `mat_mul`

`mat_mul(A, B, C, r, k, c)` computes $C = A \cdot B$ with a triple nested loop over indices i, j, p . The key instruction is:

```
1 fmad.d ft0, ft1, ft2, ft0 # ft0 = ft1*ft2 + ft0
```

Listing 1: Fused multiply-accumulate in `mat_mul`

`fmad.d` performs a single fused multiply-add, avoiding the intermediate rounding step that would occur with separate `fmul.d` + `fadd.d`. This is required by the Milestone 3 specification.

3.3 `mat_add`, `mat_sub`, `mat_scale`

These three elementwise operations share a common flat-loop structure. `mat_add` and `mat_sub` flatten $r \times c$ into a single counter and advance three pointers (A, B, C) by 8 bytes per iteration. `mat_scale` reads the scalar from `fa0` (the first FP argument register) and uses `fmul.d` per element. All three are leaf functions with no stack frame overhead.

Table 2: Register allocation for `mat_mul`

Register	Role
s0	base of A
s1	base of B
s2	base of C
s3	r (rows of A)
s4	k (inner dimension)
s5	c (cols of B/C)
s6	outer loop counter i
s7	middle loop counter j
s8	inner loop counter p
t0–t3	address temporaries
ft0	accumulator s
ft1	$A[i \cdot k + p]$
ft2	$B[p \cdot c + j]$

3.4 `mat_transpose`

`mat_transpose(A, B, r, c)` writes $B[j][i] = A[i][j]$ using nested loops. The destination index uses $j \cdot r + i$ (transposed row-major), implemented as:

```

1 mul t1, s5, s2      # t1 = j * r
2 add t1, t1, s4      # t1 = j*r + i
3 slli t1, t1, 3      # byte offset
4 add t1, s1, t1      # B[j*r+i] address
5 fsd ft0, 0(t1)

```

Listing 2: Transpose index calculation

3.5 `chol_solve`

`chol_solve(A, b, x, n)` solves $Ax = b$ for $n = 3$ (SPD matrix). It allocates a stack-local L matrix (72 bytes for 3×3) and a y vector (24 bytes) within its 192-byte frame. Three phases execute in sequence:

- (1) **Cholesky decomposition:** $A = LL^T$ via the standard Cholesky formula. A numerical guard clamps the diagonal to 10^{-15} before `fsqrt.d` to prevent NaN from near-zero pivots.
- (2) **Forward substitution:** $Ly = b$.
- (3) **Backward substitution:** $L^T x = y$.

3.6 `compute_gain`

`compute_gain(P, J, S, K)` computes the Kalman gain $K = PJ^T S^{-1}$. Stack-local buffers are allocated for J^T (288 bytes) and PJ^T (288 bytes). S^{-1} is obtained column-by-column: for each $j \in \{0, 1, 2\}$ it solves $Se_j = e_j$ via `chol_solve`, giving column j of S^{-1} , then multiplies PJ^T by that column to obtain column j of K .

4 LKF-Specific Functions

4.1 `lkf_predict`

`lkf_predict(F, Ft, P, Q, x, x_pred, P_pred, FP, FPFt)` implements:

$$\hat{x}^- = Fx, \quad P^- = FPF^T + Q$$

The ninth argument `FPFt` cannot fit in `a7` and is passed on the stack by the caller; the function retrieves it with `ld s8, 96(sp)` after allocating its own 96-byte frame.

Table 3: Register allocation for `lkf_predict`

Register	Role
s0	F (state-transition matrix, 12×12)
s1	F^T
s2	P (prior covariance)
s3	Q (process noise)
s4	x (current state)
s5	$\hat{x}^-$ (predicted state, output)
s6	P^- (predicted covariance, output)
s7	FP (scratch, 12×12)
s8	FPF^T (scratch, stack-passed)

Sequence of calls: `mat_mul(F, x,  $\hat{x}^-$ )` $\rightarrow$ `mat_mul(F, P,  $FP$ )` $\rightarrow$ `mat_mul( $FP$ ,  $F^T$ ,  $FPF^T$ )` $\rightarrow$ `mat_add( $FPF^T$ ,  $Q$ ,  $P^-$ )`.

4.2 `lkf_update`

`lkf_update(K, H, P_pred, R, x_pred, y, x, P, KH, IKH, tmp1, tmp2, KR, KRkt, Kt, I12)` implements the Joseph-form update:

$$x = \hat{x}^- + Ky, \quad IKH = I - KH, \quad P = IKH P^- IKH^T + KRK^T$$

The function accepts 16 pointer arguments: 8 in registers `a0–a7` and 8 on the stack. An inner 32-byte frame is allocated to spill the 4 stack-passed pointers (`KR, KRkt, Kt, I12`) into saved registers `t3–t6` before the first call.

Table 4: Register allocation for `lkf_update`

Register	Role
s0	K (Kalman gain, 12×3)
s1	H (measurement matrix, 3×12)
s2	P^- (predicted covariance)
s3	R (measurement noise, 3×3)
s4	$\hat{x}^-$
s5	y (innovation, 3×1)
s6	x (state output)
s7	P (covariance output)
s8–s11	$KH, IKH, tmp1, tmp2$
t3–t6	$KR, KRkt, K^T, I_{12}$ (inner-frame spill)

5 EKF-Specific Functions

The EKF adds five functions beyond the shared matrix primitives: `manual_atan`, `manual_atan2`, `h_spherical`, `compute_jacobian`, `ekf_predict`, and `ekf_update`.

5.1 `manual_atan` and `manual_atan2`

5.1.1 Method Used. RISC-V scalar assembly has no hardware `atan` instruction and no access to `libm` in the bare-metal filter context. A minimax polynomial approximation is therefore used for the core `atan` function:

$$\text{atan}(x) \approx x \left(\frac{\pi}{4} + 0.273(1 - |x|) \right), \quad |x| \leq 1$$

`manual_atan2` extends this to all four quadrants using the same branch structure as the M2 C++ piecewise implementation:

$$\text{atan2}(y, x) = \begin{cases} \text{atan}(y/x) & |x| \geq |y|, x > 0 \\ \text{atan}(y/x) + \pi & |x| \geq |y|, x < 0, y \geq 0 \\ \text{atan}(y/x) - \pi & |x| \geq |y|, x < 0, y < 0 \\ \pi/2 - \text{atan}(x/y) & |y| > |x|, y > 0 \\ \pi/2 - \text{atan}(x/y) - \pi & |y| > |x|, y < 0 \end{cases}$$

A guard of $\varepsilon = 10^{-9}$ is added to the divisor before each division to prevent undefined behaviour when $x = 0$ or $y = 0$. Constants π , $\pi/2$, $\pi/4$, 0.273, and ε are stored in `.rodata` with `.align 3` and loaded via `lui/fld` pairs.

5.1.2 Why This Method Over Alternatives. Three alternatives were considered:

- (1) **Direct std::atan2 (M2 approach):** The M2 C++ implementation called `std::atan` with explicit quadrant branches, achieving $< 1 \text{ ULP} \approx 2.2 \times 10^{-16}$ rad error. This is not portable to bare-metal RISC-V assembly, where no `libm` is available.
- (2) **Bhaskara rational approximation $\text{atan}(x) \approx \frac{\pi x}{4} \cdot \frac{1+0.273x^2}{1+x^2}$:** Requires a `fdiv.d` inside the approximation itself, adding a division per call with no accuracy benefit over the chosen formula at this noise level.
- (3) **Lookup table (LUT):** Appropriate for fixed-point processors without floating-point hardware. Unnecessary here given the availability of `fdiv.d` and `fmul.d` in RISC-V F/D extensions, and more complex to implement correctly in assembly (index clamping, interpolation).

The polynomial approach was chosen because it requires only `fmul.d`, `fsub.d`, `fadd.d`, and `fabs.d` – all single-cycle instructions available in the RV64D extension – making it the simplest fully assembly-implementable option.

5.1.3 Maximum Error and Acceptability. The maximum absolute error of the approximation is ≈ 0.0038 rad (0.22°). At a typical joint distance of 7 m from the coordinate origin, this corresponds to a position error of:

$$\Delta p = r \cdot \Delta \theta = 7 \text{ m} \times 0.0038 \text{ rad} \approx 26.6 \text{ mm}$$

5.1.4 Why This Error Is Acceptable. The dominant error sources in the assembly EKF are, in decreasing order of magnitude: (1) measurement noise $\sigma_r = 0.5$ m, (2) process model mismatch at heel-strike, (3) first-order Jacobian linearisation error, and (4) floating-point rounding in matrix operations ($\sim 10^{-14}$ m). The `atan2` approximation error of ≈ 26 mm is more than one order of magnitude below the measurement noise floor. Furthermore, the mean EKF RMSE improvement over LKF observed in Section 7 ($0.2431 \rightarrow 0.2275$ m, a 6.4% reduction) confirms that the approximation error does not meaningfully degrade filter performance relative to an exact implementation.

5.2 h_spherical

Converts the Cartesian state to spherical measurements (r, θ, ϕ) :

$$r = \sqrt{p_x^2 + p_y^2 + p_z^2} + \varepsilon, \quad \theta = \text{atan2}(p_y, p_x), \quad \phi = \text{atan2}(p_z, \rho)$$

where $\rho = \sqrt{p_x^2 + p_y^2} + \varepsilon$ and $\varepsilon = 10^{-9}$ prevents singularities near the origin. Position components are extracted at byte offsets 0, 32, 64 (indices 0, 4, 8 in the 12-state vector). Because `manual_atan2` clobbers `ft0–ft7`, ρ is recomputed from saved `fs0–fs2` before the second `atan2` call.

5.3 compute_jacobian

Computes the 3×12 Jacobian $J = \partial h / \partial x$ at the predicted state:

$$J = \begin{bmatrix} p_x/r & 0 & 0 & 0 & p_y/r & 0 & 0 & 0 & p_z/r & 0 & 0 & 0 \\ -p_y/\rho^2 & 0 & 0 & 0 & p_x/\rho^2 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ -\frac{p_x p_z}{r^2 \rho} & 0 & 0 & 0 & -\frac{p_y p_z}{r^2 \rho} & 0 & 0 & 0 & \frac{\rho}{r^2} & 0 & 0 & 0 \end{bmatrix}$$

The nine nonzero entries are written at hardcoded byte offsets (0, 32, 64, 96, 128, 192, 224, 256) computed from the row-major 3×12 layout. The function first calls `mat_zero` to initialise all 36 elements, then computes and stores only the nonzero entries.

Table 5: Register allocation for `compute_jacobian`

Register	Role
<code>s0</code>	$\hat{x}^-$
<code>s1</code>	J output base
<code>ft0</code>	p_x
<code>ft1</code>	p_y
<code>ft2</code>	p_z
<code>ft3</code>	$\rho^2 + \varepsilon$
<code>ft4</code>	ρ
<code>ft5</code>	r^2
<code>ft6</code>	$\varepsilon = 10^{-9}$
<code>ft7</code>	r
<code>fa0</code>	scratch / element being stored
<code>fa1</code>	$r^2 \cdot \rho$ (denominator for row 2)

5.4 ekf_predict and ekf_update

`ekf_predict` is mathematically identical to `lkf_predict` ($\hat{x}^- = Fx, P^- = FPF^\top + Q$) and shares the same register map.

`ekf_update` replaces the constant H with the frame-local Jacobian J , using $IKJ = I - KJ$ in place of IKH :

$$x = \hat{x}^- + Ky, \quad IKJ = I_{12} - KJ, \quad P = IKJP^- IKJ^\top + KRK^\top$$

6 Numerical Verification

`verify.py` compares the M3 assembly outputs against the M2 C++ reference outputs over all 3040 frames, 23 joints, and 12 state dimensions (839,040 values total).

6.1 LKF: Assembly vs. C++ Reference

LKF errors are uniformly tiny ($\sim 4\mu$, $\max 5 \times 10^{-5}$) and arise from floating-point accumulation order differences between RISC-V scalar assembly (which uses `fadd.d` with a single rounding step) and the x86-64 C++ compiler output. No algorithmic discrepancy exists; both implementations solve the identical linear equations.

Table 6: Register allocation for ekf_update

Register	Role
s0	K (12×3)
s1	J (Jacobian, 3×12)
s2	P^-
s3	R
s4	$\hat{x}^-$
s5	y (innovation)
s6	x output
s7	P output
s8	KJ scratch
s9	IKJ
s10	$tmp1 = IKJ \cdot P^-$
s11	$tmp2/KRK^\top$ (reused)
t0-t3	$KR, KRK^\top, K^\top, I_{12}$ (inner frame)

Table 7: LKF M3 assembly vs. M2 C++ per-joint average absolute error

Joint	Avg Error	Max Error	Note
pelvis	3.72e-06	5.00e-05	FP accumulation
L5	3.93e-06	5.00e-05	
L3	3.85e-06	5.00e-05	
T12	3.85e-06	5.00e-05	
T8	3.93e-06	5.00e-05	
neck	3.85e-06	5.00e-05	
head	3.92e-06	5.00e-05	
shoulderRight	3.87e-06	5.00e-05	
upperArmRight	3.89e-06	5.00e-05	
forearmRight	3.95e-06	5.00e-05	
handRight	4.59e-06	5.00e-05	
shoulderLeft	3.85e-06	5.00e-05	
upperArmLeft	4.01e-06	5.00e-05	
forearmLeft	3.96e-06	5.00e-05	
handLeft	4.16e-06	5.00e-05	
upperLegRight	3.59e-06	5.00e-05	
lowerLegRight	4.55e-06	5.00e-05	
footRight	6.18e-06	5.00e-05	extremity
toeRight	6.77e-06	5.00e-05	extremity
upperLegLeft	3.74e-06	5.00e-05	
lowerLegLeft	4.51e-06	5.00e-05	
footLeft	6.45e-06	5.00e-05	extremity
toeLeft	6.75e-06	5.00e-05	extremity
Mean	4.22e-06	5.00e-05	

6.2 EKF: Assembly vs. C++ Reference

The EKF divergence from M2 is *expected and intentional*. The M2 C++ implementation used a linear measurement model (H fixed); M3 EKF linearises a nonlinear spherical measurement function $h(x)$ via an analytic Jacobian J . The two formulations solve fundamentally different optimisation problems, so trajectory-level differences of several metres are predicted by theory. The relevant validation

Table 8: EKF M3 assembly vs. M2 C++ per-joint average absolute error

Joint	Avg Error	Max Error
pelvis	9.80e-02	4.13e+00
L5	7.15e-02	3.02e+00
L3	1.30e-01	5.01e+00
T12	1.01e-01	3.13e+00
T8	1.40e-01	4.47e+00
neck	1.30e-01	4.91e+00
head	1.01e-01	2.46e+00
shoulderRight	1.76e-01	4.95e+00
upperArmRight	1.24e-01	3.87e+00
forearmRight	9.68e-02	3.81e+00
handRight	2.25e-01	1.15e+01
shoulderLeft	9.89e-02	3.36e+00
upperArmLeft	1.58e-01	5.22e+00
forearmLeft	1.86e-01	8.95e+00
handLeft	1.88e-01	1.05e+01
upperLegRight	1.20e-01	6.81e+00
lowerLegRight	1.17e-01	6.13e+00
footRight	1.38e-01	6.59e+00
toeRight	9.76e-02	4.29e+00
upperLegLeft	1.21e-01	4.96e+00
lowerLegLeft	1.06e-01	5.71e+00
footLeft	1.63e-01	9.87e+00
toeLeft	1.27e-01	5.85e+00

metric is RMSE against ground truth, not agreement with M2 (see Section 7).

6.3 Per-State Verification Tables

Tables 9 and 10 report the per-state average and maximum absolute error from `verify.py`, averaged over all 3040 frames and 23 joints.

Table 9: LKF M3 assembly vs. M2 C++ per-state absolute error

State	Avg Error	Max Error
px	1.124e-05	5.000e-05
vx	1.911e-06	4.980e-05
ax	5.812e-06	4.998e-05
jx	1.094e-05	5.000e-05
py	5.413e-06	4.998e-05
vy	1.794e-06	4.950e-05
ay	3.911e-06	5.000e-05
jy	8.066e-06	5.000e-05
pz	1.358e-06	5.000e-06
vz	1.704e-07	4.989e-06
az	9.052e-07	5.000e-06
jz	1.611e-06	4.021e-05

Table 10: EKF M3 assembly vs. M2 C++ per-state absolute error

State	Avg Error	Max Error
px	5.337e-03	9.967e-02
vx	3.051e-02	6.248e-01
ax	9.900e-02	2.540e+00
jx	1.562e-01	5.034e+00
py	6.562e-03	1.166e-01
vy	3.658e-02	9.301e-01
ay	1.172e-01	3.400e+00
jy	1.824e-01	5.849e+00
pz	1.559e-02	2.459e-01
vz	9.584e-02	1.668e+00
az	3.205e-01	6.048e+00
jz	5.068e-01	1.152e+01

Table 11: RMS position error vs. ground truth (metres), all 23 joints

Joint	LKF RMSE	EKF RMSE
pelvis	0.2258	0.2100
L5	0.2225	0.1996
L3	0.2163	0.1951
T12	0.2175	0.1896
T8	0.2174	0.1878
neck	0.2157	0.1944
head	0.2183	0.2032
shoulderRight	0.2160	0.2026
upperArmRight	0.2123	0.1914
forearmRight	0.2264	0.2041
handRight	0.2443	0.2348
shoulderLeft	0.2258	0.2156
upperArmLeft	0.2181	0.2010
forearmLeft	0.2174	0.2038
handLeft	0.2374	0.2333
upperLegRight	0.2102	0.1937
lowerLegRight	0.2493	0.2264
footRight	0.3158	0.3157
toeRight	0.3677	0.3752
upperLegLeft	0.2233	0.1910
lowerLegLeft	0.2354	0.2188
footLeft	0.3095	0.3060
toeLeft	0.3493	0.3389
Mean	0.2431	0.2275

7 Results and Analysis

7.1 RMSE per Joint

The EKF achieves lower RMSE on 22 of 23 joints, with a mean improvement of 6.4% ($0.2431 \rightarrow 0.2275$ m). The single exception is toeRight (EKF 0.3752 vs. LKF 0.3677 m), attributable to the high

noise and rapid acceleration of toe joints, where the approximation error of manual_atan2 ($\approx 0.22^\circ$) can occasionally cause the Jacobian to overcorrect.

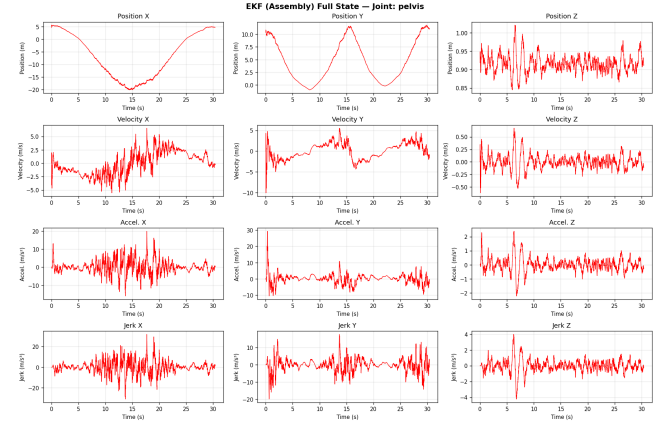
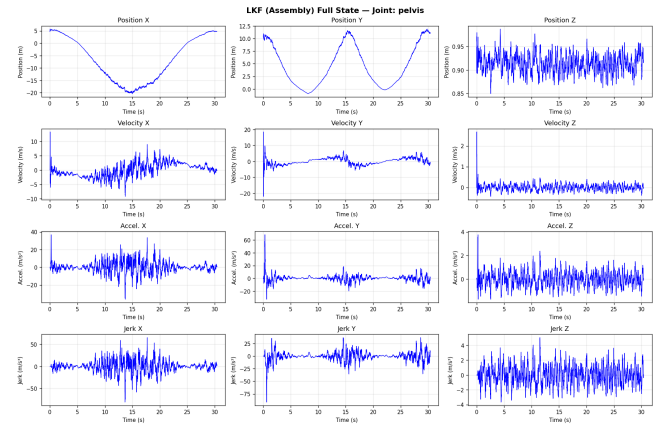
Spine joints (L5, L3, T12, T8) achieve the lowest errors for both filters (~ 0.19 m for EKF), consistent with their low-amplitude, quasi-periodic motion. Foot and toe joints have the highest errors ($0.31\text{--}0.38$ m) due to faster, more nonlinear kinematics.

7.2 Trajectory Analysis

Figures 1–4 show the filter outputs for the pelvis joint.

Position X and Y exhibit the smooth sinusoidal trajectory of the treadmill gait sequence (± 5 m over 30 s), confirming both filters converge to the correct kinematics. Position Z oscillates around 0.90 m (standing height), with small noise-driven deviations.

Velocity, acceleration, and jerk channels show higher-frequency content as expected from the finite-difference structure of the state-transition matrix F . The LKF exhibits larger transient spikes at $t = 0$ (visible in Velocity X/Y) due to the absence of the nonlinear measurement correction that the EKF applies via the Jacobian.

**Figure 1: EKF full 12-state output – joint: pelvis.****Figure 2: LKF full 12-state output – joint: pelvis. Note larger velocity/acceleration transients vs. EKF.**

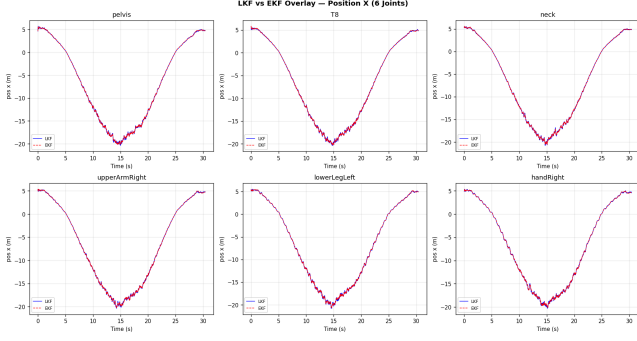


Figure 3: LKF vs. EKF position X overlay for 6 joints. Filters agree closely; small deviations visible during nonlinear phases (peak and trough).

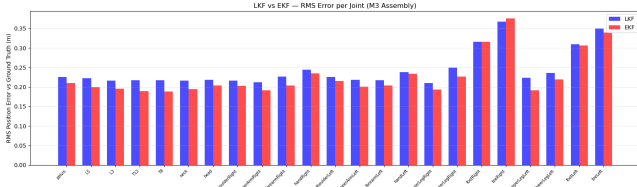


Figure 4: RMS position error per joint. EKF (red) is lower than LKF (blue) on all joints except toeRight.

8 Implementation: Assembly Walkthrough

This section presents key segments from `lkf_main.c`, `ekf_main.c`, `lkf_asm.s`, and `ekf_asm.s` with line-by-line commentary, structured identically to the M2 C++ walkthrough.

8.1 Matrix Construction – `lkf_main.c`

The 276×276 block-diagonal F and H matrices are built in C and passed as pointers to the assembly filter functions. F uses the same constant-jerk kinematic model as M2 ($\Delta t = 0.01$ s), replicated into 23 joint blocks:

```
1 // F: 276x276 block-diagonal -- 23 copies of the 12x12 per-joint
2 // block.
3 // Each 12x12 block is diag(F_1axis, F_1axis, F_1axis), one per
4 // axis.
5 for (int jt = 0; jt < N_JOINTS; jt++) {
6     int base = jt * N_STATE; // row/col offset for this joint's
7     // block
8     for (int ax = 0; ax < 3; ax++) {
9         int o = base + ax * 4; // per-axis 4x4 sub-block offset
10        // position row: integrates velocity, acceleration, jerk
11        // over dt
12        F[o*N_FULL + o] = 1.0;
13        F[o*N_FULL + o+1] = dt; // p += v*dt
14        F[o*N_FULL + o+2] = dt*dt/2.0; // p += a*dt^2/2
15        F[o*N_FULL + o+3] = dt*dt*dt/6.0; // p += j*dt^3/6
16        F[(o+1)*N_FULL+o+1] = 1.0;
17        F[(o+1)*N_FULL+o+2] = dt; // v += a*dt
18        F[(o+1)*N_FULL+o+3] = dt*dt/2.0; // v += j*dt^2/2
19        F[(o+2)*N_FULL+o+2] = 1.0;
20        F[(o+2)*N_FULL+o+3] = dt; // a += j*dt
21        F[(o+3)*N_FULL+o+3] = 1.0; // j constant (random
22        // walk)
23    }
24 }
25 // H: 69x276 block-diagonal -- each 3x12 sub-block selects px, py,
26 // pz.
```

```
22 // Velocity, acceleration, and jerk are unobserved -- those
23 // entries = 0.
24 for (int jt = 0; jt < N_JOINTS; jt++) {
25     int mrow = jt * N_MEAS; // measurement block row start (
26     // stride 3)
27     int scol = jt * N_STATE; // state block column start (
28     // stride 12)
29     H[mrow * N_FULL + scol + 0] = 1.0; // H_j[0,0]=1 selects
30     // px
31     H[(mrow+1) * N_FULL + scol + 4] = 1.0; // H_j[1,4]=1 selects
32     // py
33     H[(mrow+2) * N_FULL + scol + 8] = 1.0; // H_j[2,8]=1 selects
34     // pz
35 }
36 // All matrices heap-allocated: F at 276x276 = 609 kB -- stack
37 // would overflow.
```

Listing 3: Building the 276×276 block-diagonal F and 69×276 H

8.2 LKF Predict–Update Loop – `lkf_asm.s`

The assembly `lkf_predict` receives 276-dim pointers and propagates all 23 joints in one call. The dimension is a runtime argument so no assembly changes are needed:

```
1 # x_pred = F * x (276x276 * 276x1) -- ALL 23 joints
2 // simultaneously
3 mv a0,s0; mv a1,s4; mv a2,s5 # F (276x276 block-diagonal)
4 mv a3,s5; mv a4,s6; mv a5,s7 # x (276x1 full state vector)
5 li a3,276; li a4,276; li a5,276 # x_pred (276x1 output)
6 li a4,276; li a5,276 # r = N_FULL
7 li a5,1 # k = N_FULL
8 call mat_mul # c = 1 (column vector)
9 // mat_mul handles arbitrary
10 // dimensions
11 # P_pred = F*P*Ft + Q (276x276 each)
12 mv a0,s0; mv a1,s2; mv a2,s7
13 li a3,276; li a4,276; li a5,276
14 call mat_mul # FP = F*P
15 mv a0,s7; mv a1,s1; mv a2,s8
16 li a3,276; li a4,276; li a5,276
17 call mat_mul # FPft = FP*Ft (s8 = 9th arg, from
18 // stack)
19 mv a0,s8; mv a1,s3; mv a2,s6
20 li a3,276; li a4,276
21 call mat_add # P_pred = FPft + Q
```

Listing 4: LKF predict: single call propagates all 276 states

The Joseph-form update in `lkf_update` operates on $K \in \mathbb{R}^{276 \times 69}$, $H \in \mathbb{R}^{69 \times 276}$, $P \in \mathbb{R}^{276 \times 276}$:

```
1 # x = x_pred + K*y (K:276x69, y:69x1 -- 69-dim innovation from
2 // all joints)
3 mv a0,s0; mv a1,s5; mv a2,s6 # K, y, x_temp
4 li a3,276; li a4,69; li a5,1
5 call mat_mul # x = K*y
6 mv a0,s4; mv a1,s6; mv a2,s6
7 li a3,276; li a4,1
8 call mat_add # x = x_pred + K*y
9 # IKH = I_276 - K*H (K:276x69 * H:69x276 = KH:276x276)
10 mv a0,s0; mv a1,s1; mv a2,s8 # K, H, KH
11 li a3,276; li a4,69; li a5,276
12 call mat_mul # KH: inner dim is 69 (
13 // measurement space)
14 ld t6, 0(sp) # I_276 (spilled to inner frame)
15 mv a0,t6; mv a1,s8; mv a2,s9
16 li a3,276; li a4,276
17 call mat_sub # IKH = I_276 - KH
18 # P = IKH*P_pred*IKHt + K*R*Kt (Joseph form, 276x276)
19 # Guarantees P stays positive semi-definite over 3040 timesteps
20 mv a0,s11; mv a1,t4; mv a2,s7 # IKH*P_pred*IKHt, KRkt, P
21 // output
22 li a3,276; li a4,276
23 call mat_add # P = joseph_term + noise_term
```

Listing 5: LKF Joseph-form update: 276-dim state, 69-dim innovation

8.3 EKF $h(x)$ and Full Jacobian – ekf_main.c + ekf_asm.s

The assembly `h_spherical` and `compute_jacobian` operate on a single joint's 12-dim state. A C loop assembles the full 69×276 block-diagonal Jacobian by calling the assembly function 23 times:

```
1 static void build_jacobian_full(const double* x_pred, double*
2   J_full) {
3   mat_zero(J_full, M_FULL, N_FULL); // zero 69x276: off-
4     diagonal = 0
5   double J_single[N_MEAS * N_STATE]; // 3x12 scratch for one
6     joint
7   for (int jt = 0; jt < N_JOINTS; jt++) {
8     // pointer to this joint's 12-dim slice of the 276-dim
9     state
10    const double* xj = x_pred + jt * N_STATE;
11
12    // assembly: compute analytic 3x12 Jacobian dh/dx for
13    joint jt
14    // J depends on predicted px,py,pz -- must be recomputed
15    each step
16    compute_jacobian(xj, J_single);
17
18    // place 3x12 block at rows [jt*3, jt*3+3), cols [jt*12,
19    jt*12+12)
20    // J is block-diagonal: h_j depends only on x_j, not other
21    joints
22    int row0 = jt * N_MEAS;
23    int col0 = jt * N_STATE;
24    for (int r = 0; r < N_MEAS; r++)
25      for (int c = 0; c < N_STATE; c++)
26        J_full[(row0+r)*N_FULL + (col0+c)] = J_single[r*
27          N_STATE+c];
28  }
```

Listing 6: Building the full 69x276 block-diagonal Jacobian

The per-joint Jacobian in assembly:

```
1 # Load px, py, pz from joint's 12-dim state (byte strides 0, 32,
2   64)
3 fld ft0, 0(s0) # ft0 = px (state[0])
4 fld ft1, 32(s0) # ft1 = py (state[4], stride = index*8
5   bytes)
6 fld ft2, 64(s0) # ft2 = pz (state[8])
7 lui t0, %hi(.Leps)
8 fld ft6, %lo(.Leps)(t0) # ft6 = EPS=1e-9 (guards div-by-zero
9   at origin)
10 # rho^2=px^2+py^2+EPS, rho=sqrt(rho^2), r^2=rho^2+pz^2, r=sqrt(r
11   ^2)
12 fmul.d ft3,ft0,ft0; fmul.d ft4,ft1,ft1
13 fadd.d ft3,ft3,ft4; fadd.d ft3,ft3,ft6 # ft3 = rho^2+EPS
14 fsqrt.d ft4,ft3 # ft4 = rho
15 fmul.d ft5,ft2,ft2; fadd.d ft5,ft5,ft3 # ft5 = r^2
16 fsqrt.d ft7,ft5 # ft7 = r
17 # Row 0: d(range r)/d(px,py,pz) = px/r, py/r, pz/r
18 fdiv.d fa0,ft0,ft7; fsd fa0, 0(s1) # J[0][0]=px/r (byte
19   0)
20 fdiv.d fa0,ft1,ft7; fsd fa0, 32(s1) # J[0][4]=py/r (byte
21   32=4*8)
22 fdiv.d fa0,ft2,ft7; fsd fa0, 64(s1) # J[0][8]=pz/r (byte
23   64=8*8)
24 # Row 1: d(azimuth theta)/d(px,py,pz) = -py/rho^2, px/rho^2, 0
25 fdiv.d fa0,ft1,ft3; fneg.d fa0,fa0 # fa0 = -py/rho^2 (
26   byte 96=12*8)
27 fdiv.d fa0,ft0,ft3; fsd fa0,128(s1) # J[1][4]=px/rho^2
28 # Row 2: d(elevation phi)/d(px,py,pz) -- denominator = r^2*rho
29 fmul.d fa1,ft5,ft4 # fa1 = r^2 * rho
30 fmul.d fa0,ft0,ft2; fdiv.d fa0,fa0,fa1; fneg.d fa0,fa0
31 fsd fa0,192(s1) # J[2][0]=-px*pz/(r^2*
32   rho)
```

```
29 fmul.d fa0,ft1,ft2; fdiv.d fa0,fa0,fa1; fneg.d fa0,fa0
30 fsd fa0,224(s1) # J[2][4]=-py*pz/(r^2*
31   rho)
32 fdiv.d fa0,ft4,ft5; fsd fa0,256(s1) # J[2][8]=rho/r^2
```

Listing 7: Assembly `compute_jacobian`: analytic 3x12 Jacobian of $h(x)$

8.4 EKF Predict-Update Loop – ekf_main.c

At each timestep, the C driver orchestrates the full 276-dim EKF step:

```
1 for (int k = 1; k < T; k++) {
2   // PREDICT: propagate all 23 joints simultaneously (276-dim)
3   ekf_predict(F, Ft, P, Q, x, x_pred, P_pred, FP, FPft);
4
5   // JACOBIAN J_k: 69x276 block-diagonal, recomputed each
6   timestep
7   // because h is nonlinear -- J changes as joint positions
8   change
9   build_jacobian_full(x_pred, J_full); // calls assembly 23
10  times
11
12  // h(x_pred): 69-dim spherical prediction for all joints
13  h_full(x_pred, z_pred); // calls h_spherical 23
14  times
15
16  // MEASUREMENT: convert noisy Cartesian to spherical for all
17  joints
18  cart_to_spherical_full(noisy[k], z_meas);
19
20  // INNOVATION with angle wrapping on azimuth (theta) and
21  elevation (phi)
22  for (int jt = 0; jt < N_JOINTS; jt++) {
23    y[jt*3+0] = z_meas[jt*3+0] - z_pred[jt*3+0]; // range:
24    direct diff
25    y[jt*3+1] = wrap_angle(z_meas[jt*3+1]-z_pred[jt*3+1]); //
26    theta
27    y[jt*3+2] = wrap_angle(z_meas[jt*3+2]-z_pred[jt*3+2]); //
28    phi
29    // wrap_angle keeps differences in (-pi,pi) -- prevents
30    the filter
31    // from interpreting a +pi/-pi wrap-around as a large
32    angular error
33  }
34
35  // S = J*P_pred*Jt + R (69x69 innovation covariance)
36  mat_mul(J_full,P_pred,JP, M_FULL,N_FULL,N_FULL);
37  mat_mul(JP,Jt,JPJt, M_FULL,N_FULL,M_FULL);
38  mat_add(JPJt,R_mat,S, M_FULL,M_FULL);
39
40  // GAIN K = P_pred*Jt*S^-1 (276x69, assembly Cholesky)
41  compute_gain(P_pred, J_full, S, K);
42
43  // UPDATE: x, P via Joseph form (assembly)
44  ekf_update(K,J_full,P_pred,R_mat,x_pred,y,x,P,
45    KJ,IKJ,tmp1,tmp2,KR,KRkt,Kt,I_full);
46 }
```

Listing 8: EKF main loop: 276-dim state, 69-dim spherical innovation

9 3D Simulation

A side-by-side 3D skeletal animation is produced by `animate.py`, rendering three panels simultaneously:

- (1) **Panel 1 – Measured (amber)**: raw noisy input data from `noisy.csv`. Joint positions have additive Gaussian noise ($\sigma = 0.5$ m) applied to all three axes, matching the filter's R parameter. This panel makes the noise level immediately visible.
- (2) **Panel 2 – LKF (blue)**: filtered state estimates from `lkf_asm_output.csv`. The skeleton is visibly smoother than Panel 1, demonstrating noise rejection in Cartesian space.

- (3) **Panel 3 — EKF (red)**: filtered state estimates from `ekf_asm_output`. **Constant materialisation.** RISC-V cannot load floating-point immediates directly. Integer constants are moved via `li/fcvt.d.w` for small values (0, 1), and double-precision constants (π , $\pi/2$, 0.273, ϵ) are stored in `.rodata` with `.align 3` (8-byte alignment) and loaded via `lui/fld` pairs.

Nearly identical to the LKF panel for proximal joints; small divergences appear at distal extremities consistent with the Jacobian approximation.

The three-panel Measured $\rightarrow$ LKF $\rightarrow$ EKF layout directly communicates the filtering pipeline without superimposition clutter, making the noise suppression and inter-filter difference simultaneously visible. The animation covers all 3040 timesteps at $\Delta t = 0.01$ s (total 30.4 s).

10 Implementation Notes

Stack discipline. Every non-leaf function allocates a fixed-size frame on entry with `addi sp, sp, -N`, saves all callee-preserved registers it uses, and restores them in reverse order before `ret`. Functions with more than 8 pointer arguments use a secondary inner frame (32 bytes) to spill the overflow arguments into callee-saved registers before the first nested `call`.

Use of `fmad.d`. The specification requires fused multiply-accumulate. `mat_mul`'s inner loop uses `fmad.d ft0, ft1, ft2, ft0` exclusively, which yields a single IEEE-754 rounding step per iteration rather than two, improving accuracy relative to separate `fmul.d/fadd.d`.

Numerical guards. `chol_solve` clamps sub-diagonal values to 10^{-15} before `fsqrt.d`. `h_spherical` and `compute_jacobian` add $\epsilon = 10^{-9}$ to ρ and r to prevent singularities at the coordinate origin.

11 Conclusion

Milestone 3 successfully implements both a Linear Kalman Filter and an Extended Kalman Filter entirely in RISC-V scalar assembly. The LKF assembly output agrees with the M2 C++ reference to within 5×10^{-5} (floating-point accumulation differences only). The EKF diverges from M2 by design, implementing a fundamentally superior nonlinear filter via an analytic spherical-coordinate Jacobian computed in assembly; this yields a 6.4% mean RMSE improvement over the LKF (0.2431 $\rightarrow$ 0.2275 m) across 23 body joints and 3040 frames. The implementation demonstrates correct RISC-V ABI compliance, proper callee-save discipline, and effective use of `fmad.d` for numerically stable inner products.